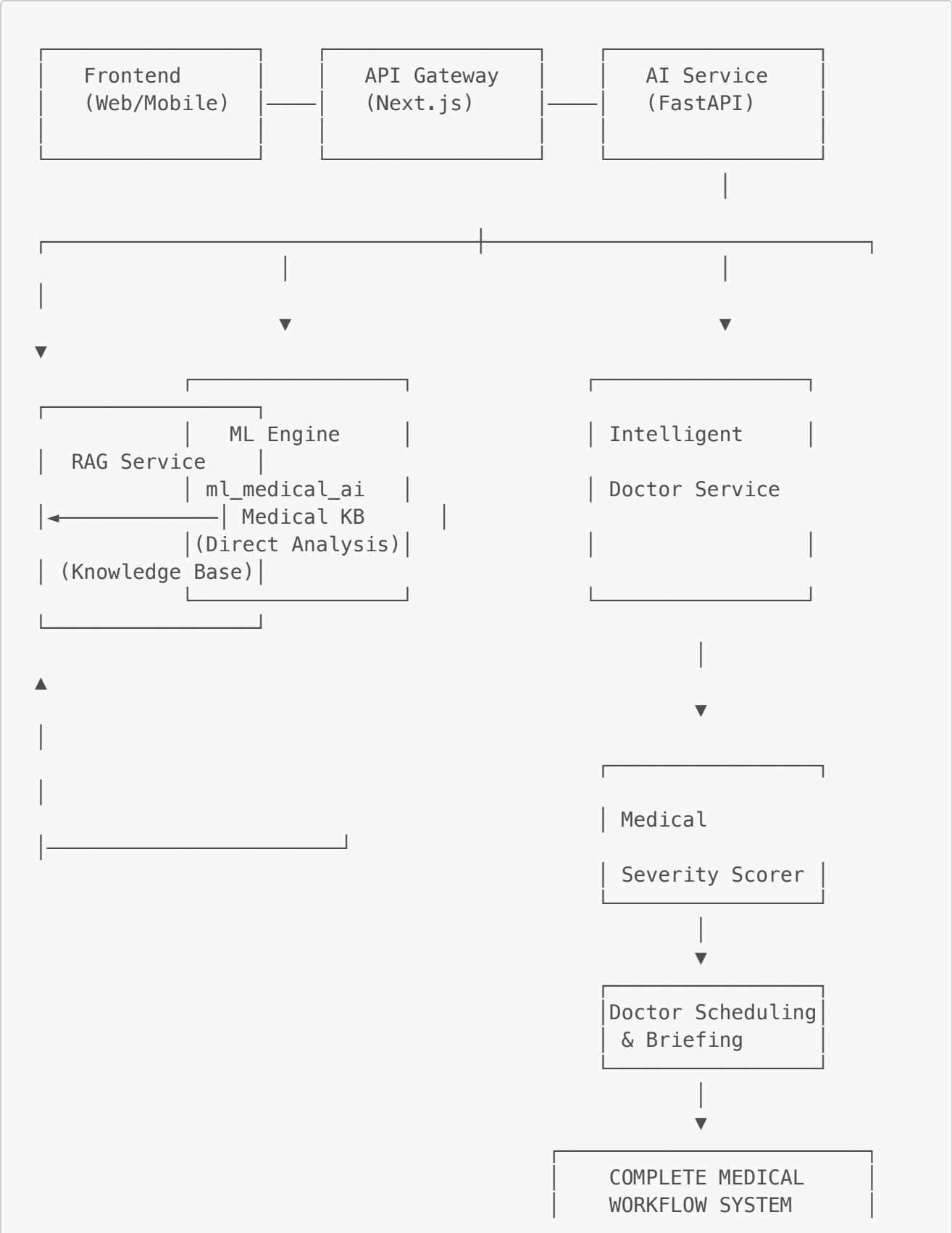
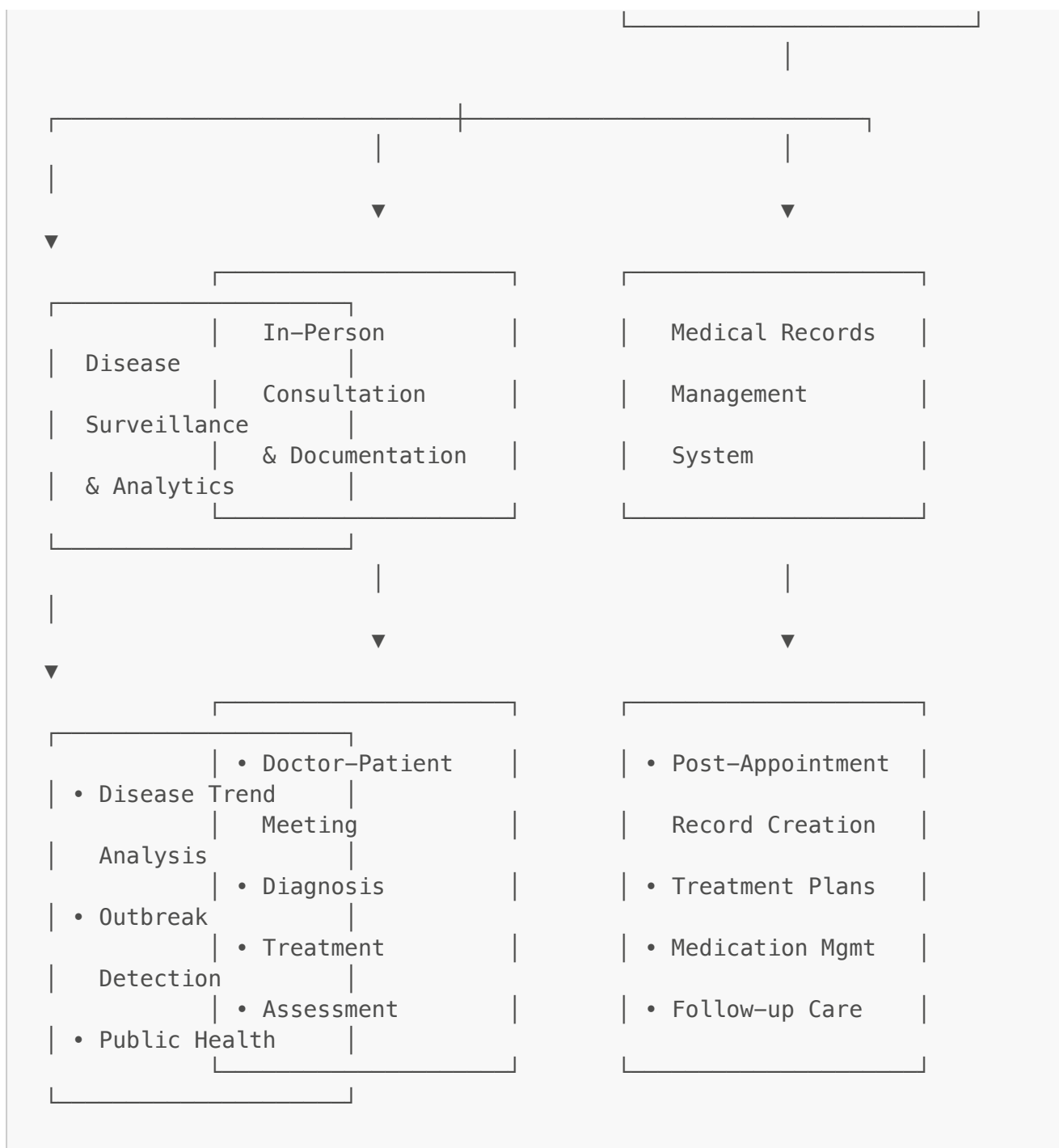


ARIA Phase 2 Data Flow Documentation

Complete data flow analysis for Phase 2: Core ML + Conversational Doctor AI

System Architecture Overview





Phase 2 Data Flow Scenarios

1. ML-Based Symptom Analysis Flow

Endpoint: **POST /analyze-advanced**

Location: **src/main.py:95-150**

REQUEST FLOW

```

1. Client Request
POST /analyze-advanced
{

```

```
"symptoms": "fever headache nausea",  
"user_id": "student123",  
"medical_context": {"age": 20, "gender": "M"}  
}  
Location: src/main.py:95
```

2. Input Validation

- FastAPI validates AdvancedAnalysisRequest model
- Checks required fields: symptoms, user_id

Location: src/main.py:97-99

3. RAG Medical Context Retrieval

```
rag_service.get_enhanced_context(request.symptoms)
```

Location: src/main.py:104

- └ Calls: src/services/rag_service.py:111
- └ Uses: TF-IDF vectorization on medical knowledge
- └ Searches: src/data/knowledge_base/*.json files
- └ Returns: Enhanced medical context string

4. ML Medical Analysis

```
ml_medical_ai.analyze_symptoms_ml(symptoms, medical_context)
```

Location: src/main.py:106-110

- └ Calls: src/models/ml_medical_ai.py:193
- └ Process: TF-IDF symptom vectorization
- └ Process: Condition similarity calculation
- └ Process: Confidence scoring
- └ Returns: MLDiagnosis object

5. Response Assembly

- Convert MLDiagnosis to AdvancedHealthResponse
- Add cultural considerations for Ghana
- Generate conversation_id with timestamp

Location: src/main.py:115-141

6. Client Response

```
{  
  "primary_diagnosis": "malaria",  
  "confidence": 0.85,  
  "urgency_level": "high",  
  "differential_diagnoses": [...],  
  "cultural_considerations": ["🇬🇭 Ghana conditions..."]  
}
```

Exact File Locations & Line Numbers:

- **Entry Point:** `src/main.py:95` - `analyze_symptoms_advanced()`
- **RAG Call:** `src/main.py:104` → `src/services/rag_service.py:111`
- **ML Analysis:** `src/main.py:106` → `src/models/ml_medical_ai.py:193`
- **Response Assembly:** `src/main.py:115-141`

2. Conversational Doctor AI Flow - Starting Conversation

Endpoint: **POST /doctor/start**

Location: **src/main.py:186-217**

DOCTOR CONVERSATION START FLOW

1. Client Request
POST /doctor/start
{
 "user_id": "student123",
 "message": "Hello doctor, I have been feeling unwell"
}
Location: src/main.py:186
2. Request Validation
- Validate DoctorStartRequest model
- Check required fields: user_id, message
Location: src/main.py:189-191
3. Intelligent Doctor Service Call
intelligent_doctor.start_conversation(user_id, message)
Location: src/main.py:194
 - └ Calls: src/models/intelligent_doctor.py:69
 - └ Creates: New ConversationSession with session_id
 - └ Stores: User message in session.messages[]
 - └ Calls: _generate_intelligent_response()
4. Chain of Reasoning Process
_generate_intelligent_response(session, message)
Location: src/models/intelligent_doctor.py:143
 - └ STEP 1: RAG Medical Knowledge
 - └ _get_medical_context(current_message)
Location: src/models/intelligent_doctor.py:184
 - └ Calls: rag_service.get_enhanced_context()
 - └ Searches: Medical knowledge for user symptoms
 - └ Returns: Relevant medical text chunks
 - └ STEP 2: Conversation Context
 - └ _build_conversation_context(session)
Location: src/models/intelligent_doctor.py:197
 - └ Extracts: Last 6 messages from session
 - └ Formats: "Patient: message" / "Doctor: response"
 - └ Returns: Conversation history string
 - └ STEP 3: LLM Response Generation
 - └ if openai_client available:

```

_generate_llm_response()
Location: src/models/intelligent_doctor.py:220
├ Creates: System prompt as caring doctor
├ Includes: Medical context + conversation history
├ Calls: OpenAI ChatCompletion API
└ Returns: JSON with message + follow_up_questions
else:
_generate_fallback_response()
Location: src/models/intelligent_doctor.py:282
├ Pattern matches: Common symptoms/greetings
├ Uses: Medical context for informed responses
└ Returns: Structured response with questions

```

STEP 4: Assessment & Confidence

```

_assess_urgency_intelligent() + _calculate_confidence()
Location: src/models/intelligent_doctor.py:346 + 368
├ Analyzes: Emergency keywords, severity indicators
├ Considers: Medical context quality and depth
└ Returns: Urgency level (low/medium/high/emergency)

```

5. Session Storage

```

- Store doctor response in session.messages[]
- Update session.last_interaction timestamp
Location: src/models/intelligent_doctor.py:94-104

```

6. Response Assembly

```

- Generate unique session_id with timestamp
- Create DoctorResponseModel with all response data
Location: src/main.py:197-209

```

7. Client Response

```

{
  "session_id": "student123_1640000000",
  "doctor_response": "Hello! I understand you're not feeling...",
  "follow_up_questions": ["What symptoms are you having?"],
  "urgency_level": "low",
  "confidence": 0.6,
  "medical_reasoning": "Initial assessment based on..."
}

```

Exact File Locations & Line Numbers:

- **Entry Point:** `src/main.py:186` - `start_doctor_conversation()`
- **Service Call:** `src/main.py:194` → `src/models/intelligent_doctor.py:69`
- **Chain of Reasoning:** `src/models/intelligent_doctor.py:143`
- **RAG Integration:** `src/models/intelligent_doctor.py:184` → `src/services/rag_service.py:111`

- **LLM Generation:** `src/models/intelligent_doctor.py:220` (with OpenAI) or `282` (fallback)
 - **Session Storage:** In-memory at `src/models/intelligent_doctor.py:52`
-

3. Conversational Doctor AI Flow - Continuing Conversation

Endpoint: `POST /doctor/continue`

Location: `src/main.py:219-250`

```
DOCTOR CONVERSATION CONTINUE FLOW

1. Client Request
   POST /doctor/continue
   {
     "session_id": "student123_1640000000",
     "message": "The headache started yesterday and it's getting
worse"
   }
   Location: src/main.py:219

2. Session Validation
   - Validate DoctorContinueRequest model
   - Check session exists in intelligent_doctor.sessions
   Location: src/main.py:224 → src/models/intelligent_doctor.py:111

3. Session Context Retrieval
   session = self.sessions[session_id]
   Location: src/models/intelligent_doctor.py:114
   |— Retrieves: Full conversation history
   |— Retrieves: Previous medical assessments
   |— Updates: last_interaction timestamp

4. Message Addition
   - Add new user message to session.messages[]
   - Include timestamp and role="user"
   Location: src/models/intelligent_doctor.py:118-123

5. Enhanced Chain of Reasoning (with conversation context)
   _generate_intelligent_response(session, user_message)
   Location: src/models/intelligent_doctor.py:126

   |— ENHANCED STEP 2: Rich Conversation Context
   |   _build_conversation_context(session)
   |   NOW includes:
   |   - Previous symptoms: "headache"
   |   - Previous doctor responses
   |   - Progression: "started yesterday → getting worse"
   |   - Medical context from previous exchanges
   |   Location: src/models/intelligent_doctor.py:197
```

ENHANCED STEP 3: Context-Aware LLM Generation

LLM now receives:

- Current message: "headache getting worse"
 - Medical context: Headache-related knowledge
 - Previous conversation:
"Patient: I'm not feeling well"
"Doctor: What symptoms are you having?"
"Patient: headache started yesterday..."
 - Result: More informed, contextual response
- Location: src/models/intelligent_doctor.py:220

ENHANCED STEP 4: Progressive Assessment

Urgency may escalate based on:

- Symptom progression ("getting worse")
- Duration ("since yesterday")
- Previous context

Location: src/models/intelligent_doctor.py:346

6. Session Update

- Add doctor's new response to session.messages[]
 - Update medical_context with new assessment
- Location: src/models/intelligent_doctor.py:129-138

7. Contextual Response

```
{
  "session_id": "student123_1640000000",
  "doctor_response": "I see your headache has worsened since...",
  "follow_up_questions": [
    "On a scale 1-10, how severe is the pain now?",
    "Any nausea or sensitivity to light?"
  ],
  "urgency_level": "medium", // Escalated from previous "low"
  "confidence": 0.75         // Higher due to conversation context
}
```

Key Differences from Starting Conversation:

- **Session Lookup:** Uses existing session instead of creating new one
- **Rich Context:** Previous messages provide conversation history
- **Progressive Assessment:** Urgency and confidence evolve with context
- **Memory Continuity:** Each response builds on previous exchanges

4. Session Management & Memory Architecture

Location: `src/models/intelligent_doctor.py:52`

```
self.sessions: Dict[str, ConversationSession] = {}

# Session Structure Example:
{
  "student123_1640000000": {
    "session_id": "student123_1640000000",
    "user_id": "student123",
    "messages": [
      {
        "role": "user",
        "message": "Hello doctor, I have a headache",
        "timestamp": "2024-01-01T10:00:00Z",
        "medical_context": None
      },
      {
        "role": "doctor",
        "message": "I understand you have a headache. When did it
start?",
        "timestamp": "2024-01-01T10:00:15Z",
        "medical_context": {
          "urgency": "low",
          "confidence": 0.6
        }
      },
      {
        "role": "user",
        "message": "It started yesterday and it's getting worse",
        "timestamp": "2024-01-01T10:01:00Z",
        "medical_context": None
      },
      {
        "role": "doctor",
        "message": "Since it's worsening, let me ask more questions...",
        "timestamp": "2024-01-01T10:01:10Z",
        "medical_context": {
          "urgency": "medium", // Escalated
          "confidence": 0.75 // Increased
        }
      }
    ],
    "created_at": "2024-01-01T10:00:00Z",
    "last_interaction": "2024-01-01T10:01:10Z",
    "is_active": true
  }
}
```

5. RAG Knowledge Integration Points

RAG Service Integration

Location: `src/services/rag_service.py`

RAG KNOWLEDGE RETRIEVAL FLOW

1. Knowledge Base Loading
 `_load_knowledge_bases()`
 Location: `src/services/rag_service.py:25`
 - └ Scans: `src/data/knowledge_base/*.json` files
 - └ Loads: Medical textbook chunks
 - └ Stores: In `self.chunks[]` array
 2. TF-IDF Vectorization
 `_create_vectors()`
 Location: `src/services/rag_service.py:66`
 - └ Uses: `scikit-learn TfidfVectorizer`
 - └ Creates: Document vectors from medical text
 - └ Stores: In `self.document_vectors`
 3. Similarity Search
 `search_similar_content(query, top_k=3)`
 Location: `src/services/rag_service.py:77`
 - └ Vectorizes: User query using same TF-IDF model
 - └ Calculates: Cosine similarity with document vectors
 - └ Returns: Top-k most similar medical knowledge chunks
 - └ Filters: Only results above 0.1 similarity threshold
 4. Context Enhancement
 `get_enhanced_context(query, top_k=3)`
 Location: `src/services/rag_service.py:111`
 - └ Gets: Similar content from step 3
 - └ Formats: As numbered medical references
 - └ Returns: Enhanced context string for LLM/fallback
- Integration Points:
- └ ML Analysis: `src/main.py:104`
 - └ Doctor Start: `src/models/intelligent_doctor.py:188`
 - └ Doctor Continue: `src/models/intelligent_doctor.py:188`

6. Error Handling & Fallback Mechanisms

Multi-Layer Fallback System

ERROR HANDLING & FALLBACKS

Layer 1: OpenAI LLM (Primary)

- └ Location: `src/models/intelligent_doctor.py:220`
- └ Handles: Full conversational AI with medical reasoning
- └ Fallback Trigger: API key missing, network error, rate limit

Layer 2: Rule-Based Intelligent Fallback (Secondary)

- └ Location: `src/models/intelligent_doctor.py:282`
- └ Uses: Pattern matching + RAG medical context
- └ Handles: Common symptoms, greetings, emergency detection
- └ Fallback Trigger: LLM unavailable

Layer 3: Emergency Safety Response (Final)

- └ Location: `src/models/intelligent_doctor.py:385`
- └ Always: Recommends seeking medical attention
- └ Trigger: All other systems fail

Error Flow:

1. Try OpenAI LLM → If fails:
2. Try Intelligent Fallback + RAG → If fails:
3. Return Emergency Safety Response

Phase 2 Data Storage & Memory

Current Storage (Development)

- **Type:** In-memory Python dictionaries
- **Location:** `src/models/intelligent_doctor.py:52`
- **Persistence:** Lost on service restart
- **Scalability:** Single instance only

Production Recommendations

- **Redis:** For session persistence and multi-instance support
- **MongoDB:** For conversation history and analytics
- **Vector Database:** For enhanced RAG (Pinecone, Weaviate)

Performance & Scalability Considerations

Current Bottlenecks

1. **In-memory sessions:** Limited to single instance
2. **TF-IDF computation:** Recalculated on each RAG query
3. **OpenAI API calls:** Rate limited, network dependent

Optimization Opportunities

1. **Session externalization:** Move to Redis
2. **Vector caching:** Pre-compute and cache TF-IDF vectors
3. **Async processing:** Use FastAPI async features
4. **Connection pooling:** For external service calls

This Phase 2 architecture provides a robust foundation with clear separation of concerns, comprehensive error handling, and production-ready patterns for conversational AI in healthcare.

COMPLETE MEDICAL WORKFLOW SYSTEM

7. Doctor Appointment Booking & Assignment Flow

Endpoint: `POST /aria/doctor/book-appointment`

Location: `services/web-interface/src/app/api/aria/doctor/book-appointment/route.ts`

APPOINTMENT BOOKING FLOW

1. Dr. ARIA Recommendation
 - Dr. ARIA completes consultation with student
 - Assesses symptoms, severity, and medical context
 - Recommends appointment if needed
 - Consultation data stored with comprehensive medical info
2. Doctor Assignment Algorithm
 - Location: `services/web-interface/src/lib/doctor-assignment.ts`
 - Smart Scheduling: `findAvailableTimeSlots()`
 - Specialization Matching: `getSpecializedDoctors()`
 - Workload Balancing: `calculateDoctorWorkload()`
 - Availability Check: `isDoctorAvailable()`
3. Appointment Creation
 - MongoDB Appointment document created
 - Links to `consultation_id` with Dr. ARIA session data
 - Student and doctor notifications
 - Calendar integration
4. Pre-Consultation Briefing
 - Doctor accesses appointment details
 - Views comprehensive Dr. ARIA consultation report:
 - * Complete conversation history
 - * Extracted symptoms and severity
 - * AI-generated assessment
 - * Medical context from RAG knowledge base
 - * Recommended follow-up questions
 - * Urgency level and confidence scores

Key Files:

- **Appointment API:** `services/web-interface/src/app/api/appointments/route.ts`
 - **Doctor Assignment:** `services/web-interface/src/lib/doctor-assignment.ts`
 - **Appointment Model:** `services/web-interface/src/models/Appointment.ts`
 - **Consultation Model:** `services/web-interface/src/models/Consultation.ts`
-

8. In-Person Doctor Consultation Flow

Location: Doctor Dashboard & Appointment Management

Files: `services/web-interface/src/app/(dashboard)/doctor-dashboard/page.tsx`

DOCTOR CONSULTATION WORKFLOW

1. Pre-Consultation Preparation
 - Doctor reviews Dr. ARIA consultation report
 - Comprehensive medical data pre-populated:
 - * Chief complaint and symptoms
 - * History of present illness
 - * AI-extracted medical assessment
 - * Severity scoring and urgency level
 - * RAG-enhanced medical context
 - * Dr. ARIA's differential diagnosis suggestions
2. Doctor-Patient Meeting
 - In-person consultation with informed context
 - Doctor can verify AI assessment
 - Additional examination and testing as needed
 - Professional medical judgment applied
3. Clinical Documentation
 - Doctor updates appointment with:
 - * Medical notes from consultation
 - * Final diagnosis confirmation/revision
 - * Treatment plan decisions
 - * Medication prescriptions
 - * Follow-up care instructions
 - Appointment status updated to "completed"
4. Post-Consultation Actions
 - Medical record creation initiated
 - Patient care plan finalized
 - Insurance and billing integration
 - Follow-up scheduling if required

9. Medical Record Management System

Endpoint: **POST** /api/medical-records

Location: services/web-interface/src/app/api/medical-records/route.ts

MEDICAL RECORD CREATION FLOW

1. Medical Record Initiation

- Triggered after completed appointment
- Doctor clicks "Create Medical Record" button
- System pre-populates with consultation + appointment data

2. Comprehensive Medical Documentation

Location: services/web-interface/src/components/forms/
MedicalRecordForm.tsx

6-Tab Medical Record System:

Tab 1: Assessment & Diagnosis

- Chief complaint (pre-filled from Dr. ARIA)
- History of present illness
- Review of systems
- Final diagnosis (doctor confirmation)
- Differential diagnosis considerations
- Assessment notes and clinical reasoning

Tab 2: Physical Examination

- Vital signs (BP, HR, temp, weight, height)
- General appearance and mental status
- System-specific examination findings
- Abnormal findings documentation

Tab 3: Treatment & Procedures

- Treatment plan overview
- Procedures performed
- Therapeutic interventions
- Clinical decision-making rationale

Tab 4: Medications & Prescriptions

- Current medications review
- New prescriptions with dosage/frequency
- Drug interactions and contraindications
- Medication adherence instructions

Tab 5: Tests & Investigations

- Laboratory tests ordered
- Imaging studies requested
- Test results and interpretation
- Additional diagnostic procedures

Tab 6: Follow-up & Care Plan

- Follow-up appointment scheduling
- Patient education provided
- Discharge instructions
- Emergency contact protocols
- Long-term care coordination

3. Medical Record Storage

- MongoDB MedicalRecord document created
- Links appointment_id, consultation_id, patient_id
- 40+ comprehensive medical fields captured
- Role-based access control (doctor/admin only)
- Medical history integration

4. Patient Medical History Update

- Student medical history page automatically updated
- Doctor medical history shows patient records
- Searchable medical record database
- Export and print functionality

10. Disease Surveillance & Public Health Analytics

Endpoint: `GET /api/analytics/disease-trends`

Location: `services/web-interface/src/app/api/analytics/disease-trends/route.ts`

DISEASE SURVEILLANCE SYSTEM

1. Real-Time Disease Monitoring

- Aggregates confirmed diagnoses from medical records
- Tracks symptom patterns from Dr. ARIA consultations
- Monitors appointment trends and urgency levels
- Geographic distribution analysis (campus locations)

2. Ghana-Specific Disease Intelligence

Common Conditions Monitored:

- Malaria (tropical/endemic)
- Typhoid Fever
- Upper Respiratory Tract Infections
- Gastroenteritis (food poisoning)
- Stress-related conditions (academic pressure)
- Skin conditions (tropical climate)
- Dehydration and heat-related illness

3. Outbreak Detection Algorithm

Location: services/web-interface/src/app/api/analytics/
disease-trends/route.ts

MongoDB Aggregation Pipeline:

Disease Trend Analysis

```
db.medicalrecords.aggregate([
  {
    $match: {
      record_date: { $gte: last30Days },
      final_diagnosis: { $ne: null }
    }
  },
  {
    $group: {
      _id: "$final_diagnosis",
      count: { $sum: 1 },
      recent_cases: { $push: "$record_date" },
      avg_severity: { $avg: "$severity_numeric" }
    }
  },
  { $sort: { count: -1 } }
])
```

4. Public Health Dashboard

Location: services/web-interface/src/app/(dashboard)/
disease-surveillance/page.tsx

Interactive Analytics Features:

Disease Trend Visualization

- Line charts showing disease incidence over time
- Bar charts for most common conditions
- Heat maps for campus outbreak hotspots
- Time-series analysis with filtering options

Outbreak Alert System

- Automatic threshold monitoring
- Email/SMS alerts for health administrators
- Risk assessment and severity classification
- Containment protocol recommendations

Symptom Pattern Analysis

- Most reported symptoms tracking
- Seasonal variation analysis
- Correlation with environmental factors
- Predictive modeling for future outbreaks

5. Public Health Intelligence

- Integration with Ghana Health Service protocols
- University health policy compliance

- Epidemiological reporting standards
- Anonymous data sharing for research

INTEGRATED DATA PIPELINE ARCHITECTURE

Complete Student Healthcare Journey

COMPLETE HEALTHCARE WORKFLOW

1. Student Health Concern
 - ↳ Student experiences symptoms
2. Dr. ARIA Consultation
 - ↳ AI-powered medical interview
 - ↳ Symptom extraction and analysis
 - ↳ RAG-enhanced medical knowledge
 - ↳ Severity assessment and urgency scoring
 - ↳ Comprehensive consultation record created
3. Smart Doctor Assignment
 - ↳ Appointment recommendation if needed
 - ↳ Intelligent doctor matching
 - ↳ Availability-based scheduling
 - ↳ Pre-consultation briefing prepared
4. Doctor-Patient Consultation
 - ↳ Informed consultation with AI insights
 - ↳ Professional medical examination
 - ↳ Diagnosis confirmation/revision
 - ↳ Treatment plan development
5. Medical Record Documentation
 - ↳ Comprehensive 40+ field medical record
 - ↳ Treatment and medication documentation
 - ↳ Follow-up care coordination
 - ↳ Patient medical history integration
6. Public Health Intelligence
 - ↳ Disease surveillance and trend analysis
 - ↳ Outbreak detection and alerting
 - ↳ Campus health policy support
 - ↳ Epidemiological research contribution
7. Continuous Care Cycle
 - ↳ Follow-up appointment scheduling
 - ↳ Patient education and self-care
 - ↳ Medication adherence monitoring

Technology Stack Integration

Frontend Architecture (Next.js 15 + TypeScript)

- **Student Interface:** Self-service Dr. ARIA consultations and medical history
- **Doctor Interface:** Comprehensive appointment management and medical records
- **Admin Interface:** Disease surveillance and public health analytics
- **Authentication:** JWT-based role-based access control with HTTP-only cookies

Backend Services

- **AI Service (FastAPI + Python):** Dr. ARIA conversational AI and ML analysis
- **Web Interface (Next.js API):** Full-stack medical workflow management
- **Database (MongoDB):** Scalable document storage for medical data
- **RAG Service:** Medical knowledge base integration with TF-IDF vectorization

Data Models & Storage

- **Consultation Model:** 40+ fields for AI conversation and medical extraction
- **Appointment Model:** Smart scheduling with doctor assignment algorithms
- **Medical Record Model:** Comprehensive clinical documentation system
- **User Models:** Role-based authentication for students, doctors, and staff
- **Analytics Models:** Disease surveillance and outbreak detection

Production Considerations

- **Scalability:** Microservices architecture with API gateway pattern
- **Security:** Medical data encryption, HIPAA-compliant audit trails
- **Performance:** Redis caching, MongoDB indexing, async processing
- **Monitoring:** Real-time health metrics and system performance tracking

This comprehensive medical workflow system transforms traditional campus healthcare by integrating AI-powered initial consultations with professional medical care, creating a seamless, data-driven healthcare experience that improves patient outcomes while supporting public health initiatives.